REACT

Interview Cheatsheet — Clean 2–3 Line Definitions

Core Concepts · Hooks · Patterns · Context API · Redux Toolkit · Performance

What's inside

1. Core React Concepts
2. Hooks — Basics
3. Hooks — Advanced & Custom
4. Component Patterns
5. Context API
6. Performance Optimization
7. React Router
8. State Management & Redux Toolkit
9. Testing
10. Modern & Server-Side React
11. **Code Example — Context API**
12. **Code Example — Redux Toolkit**
13. **Code Example — Debounced Search**

1. Core React Concepts

JSX
A syntax extension that lets you write HTML-like markup inside JavaScript, compiled by tools like Babel into `React.createElement()` calls.

Virtual DOM
A lightweight in-memory representation of the real DOM. React diffs the new virtual tree against the previous one and applies only the minimal set of real DOM updates needed.

Reconciliation
The algorithm React uses to compare the previous and next virtual DOM trees and determine the smallest set of changes to apply to the actual DOM.

Fiber
React's internal reconciliation engine that breaks rendering work into units, allowing it to pause, prioritize, and resume work — the foundation of concurrent rendering.

Component
A reusable, self-contained piece of UI defined as a function (or class) that accepts props and returns JSX describing what should render.

Props
Read-only inputs passed from a parent component to a child, used to configure or parameterize how the child renders — never mutated by the child.

State
Internal, mutable data owned by a component that persists across renders and triggers a re-render whenever it's updated.

Keys
Special string props given to list items that help React identify which items changed, were added, or removed, avoiding incorrect DOM reuse across re-renders.

Fragment (<>)
Lets a component return multiple sibling elements without wrapping them in an extra DOM node, using `<>...</>` or `React.Fragment`.

Portals
Render a child into a DOM node outside its parent hierarchy (via `createPortal`), useful for modals and tooltips that need to escape overflow/z-index constraints.

Synthetic Events
React's cross-browser wrapper around native DOM events, providing a consistent API while internally using event delegation at the root for performance.

Controlled vs Uncontrolled Components
Controlled inputs have their value driven entirely by React state; uncontrolled inputs manage their own state internally, accessed via refs when needed.

2. Hooks — Basics

useState
Adds local state to a function component, returning the current value and a setter function that triggers a re-render when called.

useEffect
Runs side effects (data fetching, subscriptions, DOM manipulation) after render, re-running when its dependency array values change; an optional cleanup function runs before the next effect or unmount.

useRef
Returns a mutable object that persists across renders without causing re-renders when changed — commonly used to access DOM nodes or store mutable values.

useMemo
Memoizes the result of an expensive computation, recalculating only when its dependencies change, to avoid unnecessary recomputation on every render.

useCallback
Memoizes a function reference itself so it doesn't change on every render, useful for stable callbacks passed to memoized child components.

useLayoutEffect
Like `useEffect` but fires synchronously after DOM mutations and before the browser paints — used when you need to measure or adjust layout before the user sees it.

useReducer
An alternative to `useState` for complex state logic, dispatching actions to a reducer function that computes the next state — similar to Redux at the component level.

useImperativeHandle
Customizes the instance value exposed to a parent when using `ref` with `forwardRef`, letting a component expose a controlled imperative API.

3. Hooks — Advanced & Custom

useContext
Reads the current value of a Context object, re-rendering the component whenever the nearest matching Provider's value changes.

Custom Hooks
Functions prefixed with `use` that encapsulate and reuse stateful logic across components (e.g. `useDebounce`, `useFetch`), built by composing built-in hooks.

useTransition

Marks a state update as non-urgent ("transition"), letting React keep the UI responsive by deprioritizing that render behind more urgent updates.

useDeferredValue

Returns a deferred version of a value that lags behind during urgent updates, useful for keeping expensive-to-render content from blocking fast user input.

useId

Generates a unique, stable ID string for accessibility attributes, consistent between server and client rendering to avoid hydration mismatches.

useSyncExternalStore

Subscribes a component to an external store (outside React state) in a way that's safe for concurrent rendering — the mechanism many state libraries use under the hood.

Rules of Hooks

Hooks must be called at the top level (never inside loops/conditions) and only from React function components or other hooks, ensuring consistent call order across renders.

4. Component Patterns

Higher-Order Component (HOC)

A function that takes a component and returns a new enhanced component, used to share cross-cutting logic like authentication checks or data fetching.

Render Props

A pattern where a component takes a function as a prop (or child) and calls it to determine what to render, sharing logic while letting the consumer control the UI.

Compound Components

A set of components that work together to form a cohesive UI (e.g. `Tabs`, `Tabs.Panel`), sharing implicit state via context rather than prop drilling.

Container / Presentational Pattern

Separates data-fetching and logic (container) from pure rendering (presentational), though hooks have largely reduced the need for this strict split.

Lifting State Up

Moving shared state to the closest common ancestor of components that need it, so they stay in sync through shared props instead of duplicated local state.

Prop Drilling

Passing data through many layers of intermediate components that don't need it themselves, just to reach a deeply nested child — a key reason Context or state libraries are used.

Error Boundaries

Class components implementing `componentDidCatch` / `getDerivedStateFromError` that catch JS errors in their child tree and render a fallback UI instead of crashing the whole app.

5. Context API

Context

A mechanism to pass data through the component tree without manually threading props at every level, ideal for global-ish data like theme, auth, or locale.

createContext

Creates a Context object with a default value, returning a `Provider` and enabling consumption via `useContext` anywhere below the Provider.

Provider

The component that supplies a Context's current value to all descendants, re-rendering every consuming component whenever that value changes.

Context Re-render Cost

Every component consuming a Context re-renders whenever its value changes, regardless of which part changed — often mitigated by splitting contexts or memoizing the value.

Context vs Redux

Context is best for infrequently changing, simple global data; Redux (or similar) suits complex, frequently updated state needing middleware, devtools, or fine-grained selectors.

6. Performance Optimization

React.memo

A higher-order component that skips re-rendering a component if its props haven't shallowly changed, useful for expensive pure components in frequently re-rendering trees.

Code Splitting (React.lazy)

Splits the app bundle so components load only when needed, using `React.lazy()` paired with `Suspense` to show a fallback while the chunk loads.

Suspense
A component that lets you display a fallback UI while its children are "not yet ready" — originally for lazy-loaded components, now extended to data fetching.

List Virtualization
Rendering only the visible subset of a long list (plus a small buffer) instead of all items, drastically reducing DOM nodes for large datasets.

Avoiding Unnecessary Re-renders
Achieved by memoizing components/callbacks/values, keeping state as local as possible, and avoiding creating new object/array/function references inline in JSX.

Key Prop Pitfalls
Using array index as a key for reorderable lists can cause React to misidentify items across renders, leading to stale state or incorrect DOM reuse.

7. React Router

BrowserRouter
Wraps the app and enables client-side routing using the HTML5 History API, keeping the UI in sync with the URL without full page reloads.

Route & Routes
`Routes` renders the first `Route` matching the current URL, defining which component displays for each path pattern.

useNavigate
A hook returning a function to programmatically navigate to a different route, replacing the older imperative `history.push` API.

useParams / useSearchParams
`useParams` reads dynamic segments from the matched URL path; `useSearchParams` reads/writes the URL's query string parameters.

Outlet
A placeholder rendered inside a parent route's element where matched nested child routes are injected, enabling layout-based nested routing.

Protected Routes
A pattern wrapping route elements with a component that checks auth state and redirects (or renders children) accordingly, guarding access to private pages.

8. State Management & Redux Toolkit

Redux
A predictable state container where the entire app state lives in one store, updated only via dispatched actions handled by pure reducer functions.

Redux Toolkit (RTK)
The official, opinionated way to write Redux logic, reducing boilerplate via `configureStore`, `createSlice`, and built-in Immer-powered "mutable" reducers.

createSlice
Generates action creators and a reducer for a piece of state in one call, letting reducers "mutate" state directly since Immer produces the immutable update under the hood.

configureStore
Sets up the Redux store with good defaults — DevTools support, thunk middleware, and dev-mode checks for mutations — replacing the older manual `createStore`.

createAsyncThunk
Generates a thunk action creator that handles async logic (like API calls) and automatically dispatches pending/fulfilled/rejected actions for each stage.

RTK Query
A data-fetching and caching layer built into Redux Toolkit that auto-generates hooks for API endpoints, handling caching, re-fetching, and loading states.

Selectors
Functions that extract and derive specific slices of state from the store, often memoized (e.g. with `reselect`) to avoid recomputation when unrelated state changes.

Provider (react-redux)
Wraps the app and makes the Redux store available to all nested components via `useSelector` / `useDispatch` without manual prop passing.

9. Testing

React Testing Library (RTL)
A testing utility focused on testing components the way users interact with them — querying by role/text/label rather than internal implementation details.

render() & screen
`render()` mounts a component into a virtual DOM for testing; `screen` provides query methods (`getByRole`, `getByText`) to find rendered elements.

fireEvent / userEvent
`fireEvent` dispatches raw DOM events; `userEvent` simulates realistic user interactions (typing, clicking) more closely matching real browser behavior.

Mocking
Replacing real dependencies (API calls, modules, timers) with controlled fake implementations using `jest.mock` so tests are fast, deterministic, and isolated.

act()

Ensures all state updates and effects related to a rendered component are processed before assertions run, preventing "not wrapped in act" warnings.

Snapshot Testing

Captures a component's rendered output and compares it to a saved snapshot on subsequent runs, flagging any unintended UI changes.

10. Modern & Server-Side React

Server-Side Rendering (SSR)

Renders the initial HTML on the server and sends it to the browser, improving perceived load time and SEO, then "hydrates" it into an interactive app on the client.

Concurrent Rendering

React's ability to prepare multiple versions of the UI in memory and interrupt/prioritize rendering work, keeping the app responsive during heavy updates.

Hydration

The process of attaching React's event listeners and internal state to server-rendered HTML on the client, making static markup interactive without re-rendering from scratch.

Next.js (Framework Context)

A React framework providing file-based routing, SSR/SSG, and API routes out of the box, commonly used as the production-grade way to build React apps.

React Server Components (RSC)

Components that render entirely on the server and send serialized output to the client, reducing client bundle size since their code never ships to the browser.

Strict Mode

A development-only wrapper that intentionally double-invokes certain functions (renders, effects) to help surface impure logic and side-effect bugs early.

Code Example — Context API (Auth Context)

A typical pattern: create a context, wrap the app in a Provider, and expose a custom hook for clean consumption.

```
// AuthContext.jsx
import { createContext, useContext, useState, useMemo } from 'react';

const AuthContext = createContext(null);

export function AuthProvider({ children }) {
  const [user, setUser] = useState(null);

  const login = async (credentials) => {
    const res = await fetch('/api/login', {
      method: 'POST',
      body: JSON.stringify(credentials),
    });
    const data = await res.json();
    setUser(data.user);
  };

  const logout = () => setUser(null);

  // Memoize value so consumers don't re-render on every provider render
  const value = useMemo(
    () => ({ user, login, logout }),
    [user]
  );

  return (
    <AuthContext.Provider value={value}>
      {children}
    </AuthContext.Provider>
  );
}

// Custom hook for clean, safe consumption
export function useAuth() {
  const ctx = useContext(AuthContext);
  if (!ctx) {
    throw new Error('useAuth must be used within an AuthProvider');
  }
  return ctx;
}

// main.jsx — wrap the app once at the root
import { AuthProvider } from './AuthContext';

<AuthProvider>
  <App />
</AuthProvider>

// LoginButton.jsx — consume anywhere below the provider
import { useAuth } from './AuthContext';

function LoginButton() {
  const { user, login, logout } = useAuth();
  return user
    ? <button onClick={logout}>Log out ({user.name})</button>
    : <button onClick={() => login({ id: 'demo' })}>Log in</button>;
}
```

Code Example — Redux Toolkit (Todos Slice + Store)

A complete slice with sync + async actions via createAsyncThunk, wired into a store and consumed with hooks.

```
// todosSlice.js
import { createSlice, createAsyncThunk } from '@reduxjs/toolkit';

// Async thunk — auto-dispatches pending/fulfilled/rejected
export const fetchTodos = createAsyncThunk(
  'todos/fetchTodos',
  async () => {
    const res = await fetch('/api/todos');
    return res.json(); // becomes action.payload
  }
);

const todosSlice = createSlice({
  name: 'todos',
  initialState: { items: [], status: 'idle', error: null },
  reducers: {
    // "Mutating" logic is safe here — Immer produces an immutable update
    todoAdded(state, action) {
      state.items.push({ id: Date.now(), text: action.payload, done: false });
    },
    todoToggled(state, action) {
      const todo = state.items.find(t => t.id === action.payload);
      if (todo) todo.done = !todo.done;
    },
  },
  extraReducers: (builder) => {
    builder
      .addCase(fetchTodos.pending, (state) => { state.status = 'loading'; })
      .addCase(fetchTodos.fulfilled, (state, action) => {
        state.status = 'succeeded';
        state.items = action.payload;
      })
      .addCase(fetchTodos.rejected, (state, action) => {
        state.status = 'failed';
        state.error = action.error.message;
      });
  },
});

export const { todoAdded, todoToggled } = todosSlice.actions;
export const selectTodos = (state) => state.todos.items;
export default todosSlice.reducer;

// store.js
import { configureStore } from '@reduxjs/toolkit';
import todosReducer from './todosSlice';

export const store = configureStore({
  reducer: { todos: todosReducer },
});

// TodoList.jsx — consuming with hooks
import { useDispatch, useSelector } from 'react-redux';
import { useEffect } from 'react';
import { fetchTodos, todoAdded, todoToggled, selectTodos } from './todosSlice';

function TodoList() {
  const dispatch = useDispatch();
  const todos = useSelector(selectTodos);

  useEffect(() => { dispatch(fetchTodos()); }, [dispatch]);

  return (
    <ul>
      {todos.map((t) => (
        <li key={t.id} onClick={() => dispatch(todoToggled(t.id))}>
          {t.done ? '    ' : '    '}{t.text}
        </li>
      ))}
    </ul>
  );
}
```

Code Example — Debounced Search (Custom Hook)

A reusable useDebounce hook combined with useEffect to avoid firing an API call on every keystroke — a common interview ask.

```
// useDebounce.js — generic reusable debounce hook
import { useState, useEffect } from 'react';

export function useDebounce(value, delay = 300) {
  const [debounced, setDebounced] = useState(value);

  useEffect(() => {
    const timer = setTimeout(() => setDebounced(value), delay);
    // Cleanup cancels the pending timeout if value changes before it fires
    return () => clearTimeout(timer);
  }, [value, delay]);

  return debounced;
}

// SearchBox.jsx — uses the debounced value to trigger the API call
import { useState, useEffect } from 'react';
import { useDebounce } from './useDebounce';

function SearchBox() {
  const [query, setQuery] = useState('');
  const [results, setResults] = useState([]);
  const [loading, setLoading] = useState(false);
  const debouncedQuery = useDebounce(query, 400);

  useEffect(() => {
    if (!debouncedQuery) {
      setResults([]);
      return;
    }

    const controller = new AbortController();
    setLoading(true);

    fetch(`/api/search?q=${debouncedQuery}`, { signal: controller.signal })
      .then((res) => res.json())
      .then((data) => setResults(data.items))
      .catch((err) => {
        if (err.name !== 'AbortError') console.error(err);
      })
      .finally(() => setLoading(false));

    // Cleanup aborts any in-flight request if query changes again
    return () => controller.abort();
  }, [debouncedQuery]);

  return (
    <div>
      <input
        value={query}
        onChange={(e) => setQuery(e.target.value)}
        placeholder="Search..."
      />
      {loading && <p>Loading…</p>}
      <ul>
        {results.map((item) => <li key={item.id}>{item.title}</li>)}
      </ul>
    </div>
  );
}
```

Prepared as a quick-revision reference for React interview prep — covers core concepts, hooks, and patterns through practical Context API, Redux Toolkit, and search implementation code.